Proyecto 1: FaroNet

Documentación del proyecto

David Zamora
Diego Zamora
Noviembre, 2026

Contenido

1. Introducción
   - Estrategia de solución
2. Ambiente de desarrollo
   - Software
   - Hardware y herramientas físicas
   - Depuración
   - Flujo de trabajo con Git
3. Diseño
   - 3.1 Hardware: el Faro
   - 3.2 Capa 1: protocolo del Faro
   - 3.3 Capas 2 y 3: la mesh FaroNet
   - 3.4 Capa 7 dentro de FaroNet
   - 3.5 Capa 7 en el edge (Cloudflare Workers)
   - 3.6 Modelo de amenazas
   - 3.7 Privacidad: requisitos del enunciado
4. Estructuras de datos y funciones principales
   - 4.1 Firmware (`firmware/faro/`)
   - 4.2 Biblioteca DispositivoLuzAdaptador (`adaptador/`)
   - 4.3 Nodo de la mesh (`nodo/faronet/`)
   - 4.4 Gateway (`gateway/faronet_gateway/`)
   - 4.5 Worker (`worker/src/`)
5. Instrucciones para ejecutar
   - 5.1 Preparar el Faro
   - 5.2 Instalar la biblioteca, el nodo y el gateway
   - 5.3 Probar el Faro
   - 5.4 Ejecutar los nodos
   - 5.5 Desplegar el Worker
   - 5.6 Usar el edge desde la terminal
   - 5.7 Pruebas automáticas
6. Actividades realizadas por estudiante
   - David Zamora
   - Diego Zamora
7. Autoevaluación
   - 7.1 Estado final
   - 7.2 Problemas encontrados y limitaciones
   - 7.3 Reporte de commits
   - 7.4 Calificación según la rúbrica
   - 7.5 Autoevaluación de aprendizaje
8. Lecciones aprendidas
9. Bibliografía
10. Documentación del código fuente

Anexo: RFC de FaroNet

> **Estado de este documento.** Versión del 2 de noviembre de 2026. Los puntos marcados como **[PENDIENTE]** son trabajo que todavía no está hecho o datos que solo el equipo puede llenar (horas, autoevaluación). Las pruebas Faro contra Faro y las mediciones de tasa y BER a distintas distancias **no se pudieron realizar** antes de la entrega (sección 7.1); no se reportan resultados que no se hayan medido.

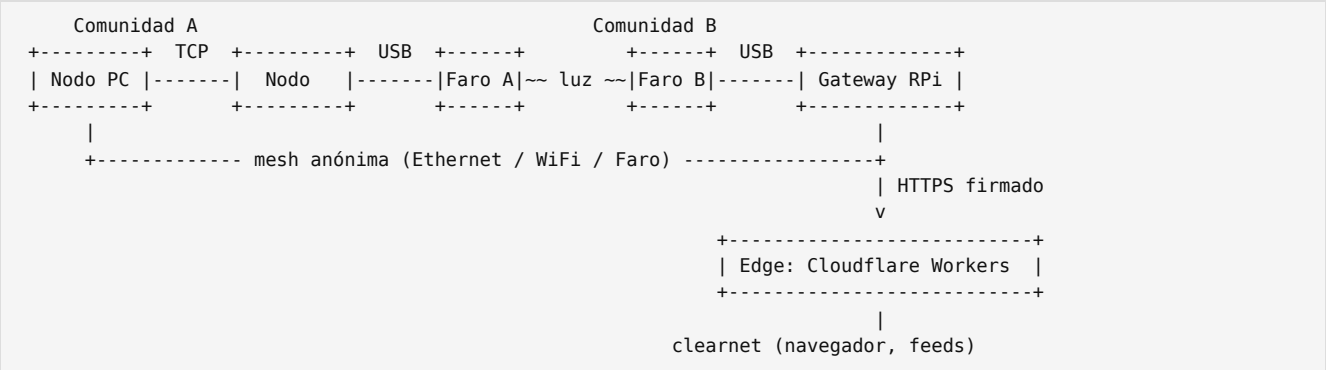
1. Introducción

Hoy cifrar el contenido de un mensaje no basta para proteger a quien lo envía. Los **metadatos** (quién habla con quién, cuándo, desde dónde y con qué frecuencia) bastan para vigilar a una persona aunque nunca se lea una sola palabra de lo que escribe. FaroNet es una red de mensajería construida desde el cobre hacia arriba cuyo objetivo es proteger también esos metadatos.

El proyecto se apoya en cuatro piezas, cada una sobre la anterior:

1. **El Faro**: un dispositivo soldado que transmite y recibe datos con luz infrarroja, conectado por USB a la computadora de un nodo.
2. **Un protocolo de enlace propio (Capa 1)**: tramas de hasta 128 bytes con CRC, acceso al medio, versión y una dirección física rotable.
3. **Una mesh anónima (Capas 2 y 3)**: enruta celdas de tamaño fijo por Ethernet, WiFi o el Faro, con circuitos onion de 3 a 5 saltos, de modo que ningún nodo intermedio conoce a la vez el origen y el destino.
4. **Una salida a Internet en el edge (Capa 7)**: un gateway que publica solo los mensajes marcados como públicos en Cloudflare Workers (salas en tiempo real, feeds Atom, buzón muerto y directorio público), sin guardar datos que identifiquen a los usuarios.

Las comunidades de FaroNet están separadas y solo se unen por un par de Faros:



Estrategia de solución

Ninguno de los dos integrantes tenía experiencia previa en electrónica, protocolos de enlace, mesh, criptografía aplicada ni serverless (ver el diagnóstico del kick-off, `docs/kickoff/Kickoff_FaroNet.docx`). Por eso se construyó **de abajo hacia arriba**, validando cada capa antes de pasar a la siguiente:

Fase	Contenido	Estado
0	Aprendizaje base (electrónica, Arduino, sockets, criptografía)	Hecho
1	Prototipo del Faro en protoboard, luz entre dos Faros	Cada Faro probado por separado; entre dos Faros **no realizado**
2	Capa 1: trama, CRC, acceso al medio, medición de tasa y BER	Protocolo hecho; medición **no realizada**
3	Faro soldado en placa perforada, pruebas de multímetro	Dos placas soldadas parcialmente (fotos en 3.1)
4	Mesh FaroNet: directorio, ruteo, onion routing	Hecho
5	Gateway (Raspberry Pi) y Cloudflare Workers	Hecho y desplegado
6	RFC, documentación y ensayo de la defensa	RFC hecho; este documento

Para no frenar el software mientras se armaba el hardware, la biblioteca incluye un **simulador del Faro** (`adaptador/dispositivo_luz_adaptador/simulador.py`) que imita el firmware y un medio con BER configurable. Así la mesh se programó y probó sin tener los Faros soldados.

2. Ambiente de desarrollo

Software

Uso	Herramienta
Sistemas operativos	GNU/Linux (Ubuntu) en las PC; Raspberry Pi OS en los nodos Raspberry Pi
Editor	Visual Studio Code
Control del Faro	Raspberry Pi por GPIO con `pigpio` (`adaptador/dispositivo_luz_adaptador/faro_gpio.py`), una Raspberry por Faro
Firmware alternativo	Arduino (C/C++) y `arduino-cli` con el core `arduino:avr`; se usó en el prototipo en protoboard
Pruebas del núcleo del firmware	`g++ -std=c++11` en la PC (`firmware/faro/test/test_core.cpp`)
Biblioteca, nodos y gateway	Python 3.12, `pyserial`, `pigpio`, PyNaCl (libsodium), Scapy (opcional)
Aplicación en el edge	TypeScript sobre Cloudflare Workers (plan gratuito), Durable Objects con SQLite, `wrangler`, Node.js 22
Integración continua	GitHub Actions (`.github/workflows/ci.yml`)
Control de versiones	Git y GitHub, una rama por funcionalidad y pull requests
Documentación	Markdown → PDF con Pandoc; RFC en texto ASCII paginado con `paginar.py`
Asistencia	Claude (Anthropic) como asistente de programación, revisión y redacción

Hardware y herramientas físicas

- Una **Raspberry Pi por Faro**, que lo maneja por sus pines GPIO. El Arduino Uno se usó en el prototipo en protoboard; el firmware sigue en el repositorio como alternativa (compila para Uno y Nano).
- Raspberry Pi 2 y Raspberry Pi 3/4 como nodos (`faronet-sur` de David y `faronet-gw` de Diego).
- Cautín, estaño con resina, malla desoldante, pinzas y soporte.
- Multímetro digital: continuidad, resistencia, ausencia de corto entre +5 V y GND antes de energizar y voltajes en puntos clave.
- Protoboard para el prototipo y placa perforada (HS-03) para la versión final.

Depuración

- **Faro**: LEDs TX/RX/estado, los comandos `faro sondear --continuo` (lee A0 y la salida del comparador) y `faro emisor encender|apagar` (LED IR fijo para medir con el multímetro), y `hardware/calibrar_receptor.py`.
- **Protocolo**: pruebas unitarias contra Faros simulados y contra el núcleo C++ del firmware; Scapy (`scapy_faro.FaroTrama`) para armar tramas a mano.
- **Mesh**: consola del nodo (`nodos`, `rutas`, `circuitos`) y `nodo/demo_local.sh` (5 relés locales). Nada se escribe a disco, por la política de no registro.
- **Edge**: `wrangler dev` local y pruebas de integración en `gateway/tests`.

Flujo de trabajo con Git

- `main` siempre estable y compilable.
- Una rama por funcionalidad (`feature/faro-capa1`, `feature/mesh-onion`, `feature/puente-edge`, `feature/despliegue-worker`, `feature/rfc`, `feature/temas-privados`, …), integrada por pull request que revisa y fusiona el otro integrante.
- Cada pull request corre la CI: pruebas de `adaptador/` (incluidas las del núcleo C++), pruebas de `nodo/`, verificación de tipos del Worker con pruebas de integración contra `wrangler dev`, y compilación del firmware para Nano y Uno.
- No se suben secretos: el Worker solo tiene llaves **públicas** de gateways en `wrangler.toml`.

Repositorio: https://github.com/DavidZamoraB/faronet

3. Diseño

Esta sección explica las decisiones. La especificación completa está en el RFC (`docs/rfc-faronet.txt`, anexo al final de este PDF).

3.1 Hardware: el Faro

El Faro tiene cinco bloques y un conector J1 de 9 pines. Lo maneja una **Raspberry Pi por GPIO** (una Raspberry por Faro), que hace el trabajo que en el diseño original hacía un Arduino:

```
              +-----------------------------------------------+
Raspberry Pi =| J1 (9 pines) + cables dupont                  |
              |  GPIO17 --> [ Emisor: 2N2222A + LED IR ]  ~~~> luz
 luz ~~~>  [ Receptor: fototransistor -> LM393 ] -> divisor -> GPIO27
              |  GPIO5 -> LED TX   GPIO6 -> LED RX   GPIO13 -> LED
              |  GPIO22 <- boton de panico                     |
              +-----------------------------------------------+
```

J1	Raspberry (BCM / pin físico)	Arduino (alternativa)	Función
B6	5 V / pin 2	5V	Alimentación
B7	GND / pin 6	GND	Tierra
B8	GPIO27 / pin 13, **por un divisor 10 kΩ / 20 kΩ**	D2	Salida del comparador
B9	no se conecta	A0	Voltaje del sensor (solo calibración)
B10	GPIO17 / pin 11	D4	Base del 2N2222A (vía R2 = 1 kΩ)
B11	GPIO22 / pin 15	D3	Botón de pánico a GND (pull-up interno)
B12 / B13 / B14	GPIO5, GPIO6, GPIO13 / pines 29, 31, 33	D5, D6, D7	LED TX (verde) / RX (azul) / estado-pánico (rojo)

Por qué Raspberry y no Arduino. El equipo decidió usar las Raspberry Pi como controladores de los Faros (el enunciado lo permite: “SBC … conexión por GPIO en el caso de una Raspberry Pi”) y desechar el Arduino. La capa 1 no cambia: `faro_gpio.py` implementa la misma codificación Manchester, CSMA/CA con ACK, CRC, dirección rotable y pánico que el firmware, y habla con el resto del software con el mismo protocolo, así que el nodo y el gateway no cambian (`faronet --faro gpio`). Los tiempos los maneja **pigpio**: cada trama sale como forma de onda por DMA y cada flanco del receptor llega con su marca de tiempo en microsegundos, de modo que la latencia de Linux no afecta la decodificación. Verificado en la Raspberry Pi 2: una trama de 114 bytes tarda 0,53 s a 2000 bit/s, lo esperado. La dirección

física y el secreto se guardan en un archivo de la SD (permisos 0600) que el pánico sobrescribe y borra.

Niveles de 3,3 V. Los pines de la Raspberry no toleran 5 V. La salida del LM393 sube a 5 V por R5, por eso pasa por un divisor (10 kΩ en serie y 20 kΩ a tierra, ≈ 2,5 V en alto) antes de GPIO27. La Raspberry no tiene entrada analógica, así que A0 no se usa. Los LEDs a 3,3 V brillan menos; el azul (Vf ≈ 3 V) casi no se ve.

Emisor. LED IR de 940 nm con R1 = 82 Ω (100 Ω en el Faro 1) y un 2N2222A como interruptor. Corriente del LED ≈ (5 − 1,2 − 0,2) V / 100 Ω ≈ 36 mA; corriente de base desde un pin de 3,3 V ≈ (3,3 − 0,7) V / 1 kΩ ≈ 2,6 mA, β forzado ≈ 14, así que el transistor satura. Se usa transistor porque un pin GPIO no debe dar más de unos 16 mA.

Receptor. Fototransistor IR (con R3 a GND) cuyo voltaje entra a un comparador LM393. El umbral es un **divisor fijo** R9 = 10 kΩ / R10 = 1 kΩ (≈ 0,45 V). R4 = 1,5 MΩ da histéresis y R5 = 10 kΩ es el pull-up de la salida de colector abierto. Se usa comparador y no solo la entrada analógica porque leer un voltaje analógico es lento (y la Raspberry no tiene ADC); el comparador entrega flancos limpios que pigpio marca con su tiempo.

Cambios que dejó la práctica (detalle en `hardware/bom.md`):

Pieza	Diseño original	Final	Razón
Umbral	Trimpot RV1 10 kΩ	Divisor fijo R9/R10	El trimpot del kit no variaba el voltaje
R4	1 MΩ	1,5 MΩ	Disponibilidad; funciona igual
R5	4,7 kΩ	10 kΩ	Disponibilidad; sirve como pull-up
R1 (Faro 1)	82 Ω	100 Ω	No había de 82 Ω; ≈ 36 mA en vez de 44 mA
C1 (Faro 1)	100 nF	10 nF	No había de 100 nF; sigue filtrando el 5 V del chip
Controlador	Arduino Uno/Nano	Raspberry Pi por GPIO	Decisión del equipo; una Raspberry por Faro
Salida D2	Directa al Arduino	Divisor 10 kΩ / 20 kΩ	Los pines de la Raspberry son de 3,3 V

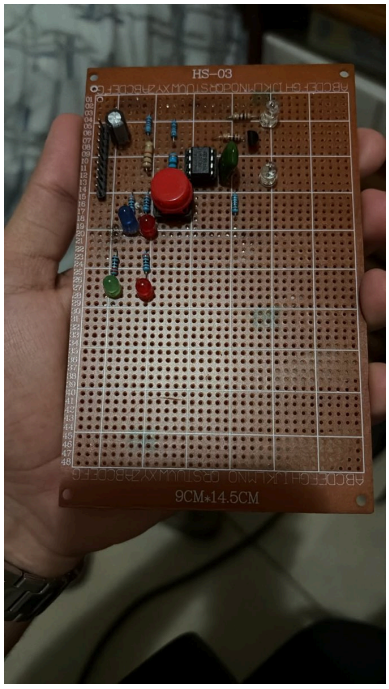
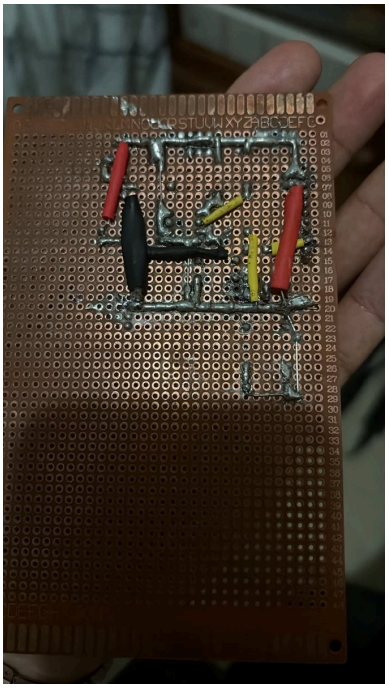
El Faro de Diego funcionó en protoboard el 20-10-2026 (A0 ≈ 55 con el LED apagado y ≈ 272 encendido). El de David pasó la prueba de recepción el 31-10-2026 (A0 de 2 a 162, frente a frente a 1 cm), pero su sensor resultó débil y el margen es justo para 1–2 m; la mejora propuesta es un fototransistor de 940 nm (PT333-3C) o R3 = 100 kΩ.

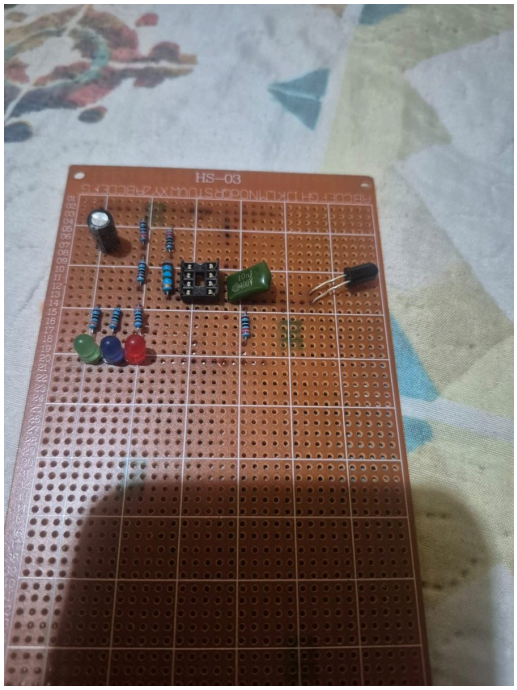
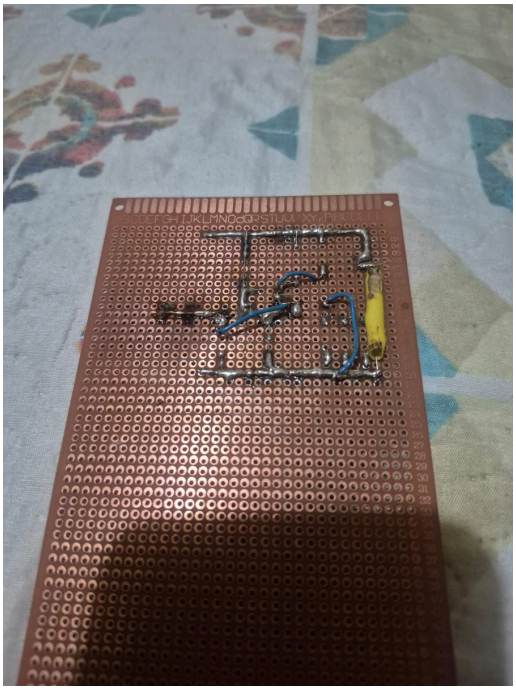
Esquemático. El esquemático por bloques, los cálculos y la netlist para KiCad están en `hardware/esquematico.md`. **[PENDIENTE]** el proyecto KiCad en `hardware/kicad/` y su exportación en PDF.

Placa perforada. El circuito se suelda en placa perforada con zócalo DIP-8 para el LM393 y un conector J1 de 9 pines macho: la Raspberry se conecta con cables dupont hembra-hembra y se retira sin desoldar (`hardware/placa_perforada.md`). Se soldaron dos Faros en placas HS-03 (9 × 14,5 cm). Las fotos de ambas caras están en `hardware/fotos/` y se muestran abajo (tomadas el 1 de noviembre de 2026).

- **Faro 1**: todo soldado y probado por partes con la Raspberry como fuente de 5 V: LEDs, emisor (el LED IR se ve en la cámara del celular), receptor con el LM393 (detecta el IR reflejado y deja de detectarlo al apagar el emisor), botón de pánico y conector J1 con 5 V y tierra. El LM393 va con la pata 7 doblada hacia afuera: esa salida no se usa y su punto quedó unido al 5 V. Se están soldando los cables de J1 a cada señal. La distribución real está en `hardware/placa/placa.py` y el diseño limpio para el Faro 2 en `placa2.py` (ambos verifican conexiones y cortos).
- **Faro 2** (David): terminadas y probadas con multímetro las etapas de alimentación (3A: rieles de 5 V y tierra, sin corto) y divisor de umbral (3B); en la foto ya están el zócalo DIP-8 (todavía sin el LM393), C1, las resistencias y LEDs indicadores y el receptor. Falta terminar el receptor (3C) y soldar el emisor (etapa 4).

[PENDIENTE] la bitácora de multímetro llena para cada Faro (`hardware/bitacora_pruebas.md`).

Faro 1: componentes	**Faro 1: cobre**
	

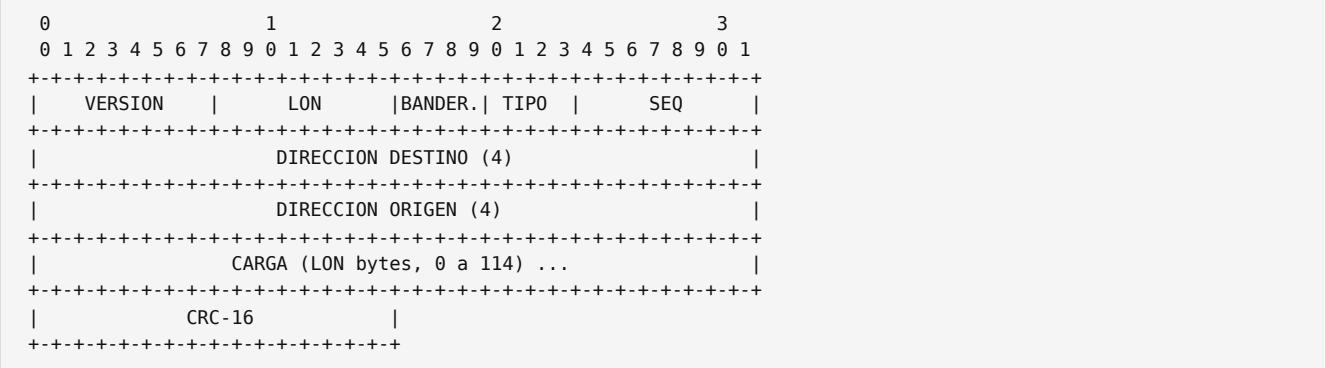
Faro 2: componentes	Faro 2: cobre
	

BOM. `hardware/bom.md`: ≈ ₡10 675 por Faro y ≈ ₡26 910 para los dos Faros más materiales compartidos (precios aproximados de octubre de 2026; se deben reemplazar por los de la factura).

3.2 Capa 1: protocolo del Faro

Decisión	Elección	Por qué
Codificación de línea	OOK con **Manchester** (IEEE 802.3)	Auto-sincronizante (corrige el desfase de reloj entre los dos Faros en cada bit), nivel promedio constante (umbral estable) y detecta errores de línea antes del CRC. Cuesta el doble de ancho de banda.
Velocidad	500–20 000 bit/s, 2 000 por defecto	Ajustable para buscar el máximo a cada distancia
Sincronización	Preámbulo 4 × `0x55` + SFD `0xD5`	Si llega el patrón negado, el receptor invierte los bits: la polaridad del hardware no importa
Trama	12 B de cabecera + 0–114 B de carga + CRC-16 = máx. 128 B	Cumple el límite del enunciado
Integridad	CRC-16/CCITT-FALSE	Detecta todos los errores de 1 y 2 bits y ráfagas de hasta 16

Decisión	Elección	Por qué
Dirección física	4 bytes en EEPROM, rotable	2^{32} direcciones bastan para decenas de Faros y ahorran 4 B por trama frente a 6
Acceso al medio	CSMA/CA con espera aleatoria exponencial (8→64 ranuras), DIFS 32 bits, SIFS 12 bits, ACK y hasta 3 reintentos	Semidúplex sin detección de colisiones; el ACK tiene prioridad (SIFS < DIFS) y se descartan duplicados
Versionado	VERSION y LON siempre en los bytes 0 y 1; la trama mide siempre 12 + LON + 2	Un receptor sabe dónde termina una trama aunque no entienda su versión
Interfaz nodo ↔ Faro	USB-serial 115 200 8N1, paquetes `COBS(tipo|datos|CRC-16)` terminados en `0x00`	Se resincroniza tras cualquier basura en la línea



El nodo puede entregar una **trama armada fuera del Faro** (por ejemplo con Scapy) con el comando `ENVIAR_CRUDA`: el Faro solo verifica que su longitud sea coherente con LON y la transmite tal cual.

3.3 Capas 2 y 3: la mesh FaroNet

- **Identidad**: cada nodo tiene una semilla de 32 bytes de la que salen una llave Ed25519 (firma) y una X25519 (onion). El ID de nodo son 8 bytes de BLAKE2b de la llave de firma.
- **Celdas de 512 bytes** siempre, con cabecera de 26 bytes (versión, comando, TTL, destino y origen *del tramo*, circuito y LON). El tamaño fijo impide que la longitud filtre información.
- **Enlaces**: TCP (Ethernet o WiFi, costo 1) y Faro (costo 4). En el Faro una celda se fragmenta en 5 tramas (4 de 112 B de datos y 1 de 64), así que el largo de las tramas tampoco revela el largo del mensaje.
- **Ruteo dinámico**: vector de distancias al estilo RIP, con horizonte dividido, envenenamiento en reversa, métrica infinita 16 y expiración a los 3 intervalos. Cada celda se reenvía al vecino más cercano al destino.
- **Directorio de nodos**: cada nodo publica una entrada firmada con Ed25519 (ID, llaves, roles, seudónimo, vigencia de 10 min) que se difunde por inundación. No hay servidor

central: una entrada alterada no verifica. Las entradas no contienen IP ni direcciones físicas.
- **Circuitos virtuales negociados**, al estilo Tor: el cliente elige 3 a 5 relés al azar y extiende el circuito salto por salto. El handshake (tipo ntor) combina dos X25519 (efímera-efímera para secreto hacia adelante y efímera-estática para autenticar al relé).
- **Capas de la cebolla**: XChaCha20-Poly1305 con nonce por contador, una capa por salto. Cada salto quita su capa y rellena con bytes aleatorios nuevos, así la celda sigue midiendo 512 y no se puede seguir comparando bytes.

El resultado es que el primer relé conoce al cliente pero no el destino, la salida conoce el destino pero no al cliente y el relé del medio no conoce a ninguno (lo comprueba la prueba `test_ningun_rele_conoce_origen_y_destino`).

3.4 Capa 7 dentro de FaroNet

- **Mensajes privados**: el texto se sella (SealedBox) hacia la llave onion del destinatario. El sobre mide siempre 391 bytes (hasta 331 de texto) y tiene TTL (24 h por defecto); los mensajes recibidos viven solo en memoria.
- **Temas privados**: un tema marcado como privado se envía a cada miembro como mensaje privado; el nombre del tema va dentro del sobre y estos mensajes nunca salen a clearnet (`publicar` sobre un tema privado da error).
- **Mensajes públicos**: el circuito termina en un nodo con rol gateway, que recibe un `MensajePublico` de tamaño fijo y lo sube al edge sin saber quién lo originó.

3.5 Capa 7 en el edge (Cloudflare Workers)

- **Relay del gateway**: el gateway firma cada petición con Ed25519 sobre `"faronet-edge-v1" | método | ruta | tiempo_ms | SHA-256(cuerpo)`. El Worker exige que la llave esté en `GATEWAYS`, que el tiempo esté a menos de 60 s y que sea estrictamente mayor que el último aceptado (anti-replay).
- **Salas públicas**: un Durable Object (SQLite) por tema con WebSockets hibernables; máximo 200 mensajes y 7 días.
- **Feed Atom** por tema en `/salas/:tema/feed.xml`.
- **Buzón muerto**: blobs de 1024 bytes cifrados con una llave derivada de un secreto compartido fuera del Worker, identificador opaco, TTL de 1 min a 72 h y lectura única atómica.
- **Directorio público**: el Worker verifica la firma de cada entrada y publica los bytes originales para que cualquiera la vuelva a verificar.
- **Rate limiting**: 60 peticiones por minuto con la clave `SHA-256(fecha | clase | IP)`; la IP no se guarda.
- **No registro**: el código no persiste `CF-Connecting-IP` ni `User-Agent`, `observability` está desactivado y las páginas usan CSP con nonce.

Despliegue: https://faronet.faronet-worker.workers.dev (PR #6). Según el README del Worker se probó de punta a punta: cliente → circuito onion de 3 saltos en la Raspberry Pi → gateway

→ Worker → sala `#general`.

3.6 Modelo de amenazas

Objetivo: ningún adversario individual debe poder saber a la vez quién envía un mensaje y a quién (o que el remitente es el autor de un mensaje público).

Adversario	Qué ve	Qué no debe ver	Mitigación
Nodo malicioso de la mesh	Su vecino anterior y siguiente, hora y cantidad de celdas	Origen y destino a la vez; el contenido	Circuitos de 3–5 saltos, celdas de 512 B, relleno nuevo por salto, cifrado extremo a extremo
Observador del medio físico (luz)	Que hay transmisión, cuándo, tamaño de trama, direcciones físicas actuales	El contenido; seguir a un Faro en el tiempo	Cifrado por capas y extremo a extremo; rotación de la dirección física con intervalo aleatorio
Operador del gateway	Mensajes públicos que salen, hora, tema y alias	El autor de los públicos; los privados	Los privados no salen de FaroNet; el gateway es la salida de un circuito y no conoce al cliente
Proveedor del edge (Cloudflare)	IP del gateway y de los navegadores, horas, tamaños y rutas (incluido el ID del buzón)	El contenido del buzón; quién originó un mensaje	Blobs cifrados de tamaño fijo con TTL; el Worker no guarda IP ni User-Agent; la clave del rate limit es un hash
Ladrón del Faro	EEPROM y memoria del Faro	Llaves e identidad, si se presionó el pánico	Botón de pánico: borra el secreto y cambia la dirección

Fuera de alcance: un adversario global que vea entrada y salida de un circuito (correlación por tiempo), varios relés que colaboren siendo primero y último salto, ataques Sybil al directorio y un adversario con acceso al USB del Faro antes del pánico.

Limitaciones conocidas (también en el RFC):

- Los enlaces TCP y Faro no van cifrados entre vecinos, así que los IDs de nodo de cada tramo viajan en claro dentro de las celdas. Por eso rotar la dirección física dificulta, pero no impide del todo, seguir a un nodo.
- El `HOLA` de los enlaces no está autenticado: un vecino puede mentir sobre su ID y desviar rutas (no rompe el anonimato porque el handshake autentica a cada salto). **[PENDIENTE]**
- No hay tráfico de cobertura: el comando `RELLENO` está reservado pero no implementado. **[PENDIENTE]**

3.7 Privacidad: requisitos del enunciado

Requisito	Cómo se cumple
Anonimato del origen	Circuitos onion de 3 a 5 saltos
Cifrado extremo a extremo	SealedBox de libsodium (PyNaCl); WebCrypto en el Worker. No hay primitivas propias
Padding a tamaño fijo	Celdas de 512 B, sobres de 391 B, blobs de 1024 B, 5 tramas por celda en el Faro
Direcciones físicas rotables	`faro direccion --rotar` y rotación automática cada 600 s ± 20 % con `os.urandom`
Botón de pánico	Sobrescribe el secreto de la EEPROM (`0xFF` y luego `0x00`), cambia la dirección, borra buffers y estadísticas, avisa al nodo (que borra llaves, circuitos, rutas, directorio y buzón) y parpadea; el LED rojo queda encendido
Sin registros	Nodos solo escriben la semilla (si se usa `--llave`); el Faro solo tiene contadores en RAM; el Worker no guarda IP ni User-Agent; todo mensaje almacenado tiene TTL

4. Estructuras de datos y funciones principales

4.1 Firmware (`firmware/faro/`)

Elemento	Descripción
`DecodificadorManchester` (`faro_core.h`)	Máquina de estados que corre dentro de la interrupción de D2: recibe el tiempo entre flancos y el nivel. Estados `RX_CAZANDO` → `RX_SINCRONIZADO` → `RX_RECIBIENDO` → `RX_LISTA`; detecta la polaridad con el SFD
`crc16`, `trama_armar`, `trama_longitud_valida`, `trama_crc_valido`, `cobs_codificar/decodificar` (`faro_core.cpp`)	Lógica sin hardware, compartida con las pruebas en la PC
`struct Estadisticas` (`faro.ino`)	Contadores: tramas TX/RX, CRC malos, sin ACK, retransmisiones, truncadas
`Visto vistos[8]`	Tabla circular (origen, SEQ) para no entregar duplicados
`emitir_trama`, `emitir_byte`	Emisión Manchester con temporización por `micros()`
`canal_ocupado`	Escucha de portadora: intervalos Manchester válidos o trama en curso
`procesar_recepcion`, `enviar_ack`, `es_duplicado`	Recepción, ACK tras SIFS y supresión de duplicados
`ejecutar_panico`, `revisar_boton_panico`	Zeroize del secreto, nueva dirección, limpieza y aviso al nodo
`cargar_eeprom`, `guardar_direccion`, `generar_direccion`	Dirección física y configuración en EEPROM

4.2 Biblioteca DispositivoLuzAdaptador (`adaptador/`)

Elemento	Descripción
`DispositivoLuzAdaptador`	Clase principal: `abrir/cerrar`, `info`, `enviar(destino, datos)` (unicast con ACK o difusión), `enviar_trama` (trama cruda), `recibir/al_recibir`, `al_panico`, `rotar_direccion`, `rotacion_automatica`, `guardar_secreto/leer_secreto`, `configurar`, `sondear`, `emisor_fijo`
`Trama` (`trama.py`)	`a_bytes()` / `desde_bytes()` con verificación de CRC; constantes `MAX_TRAMA = 128`, `MAX_CARGA = 114`, `DIFUSION`
`crc16`, `cobs.codificar/decodificar`	Iguales a las del firmware (una prueba verifica que las constantes coincidan)
`enlace.py`	Códigos de comandos, respuestas, eventos y estados del protocolo serial

Elemento	Descripción
`ResultadoTx`, `TramaRecibida`, `InfoFaro`, `Estadisticas`	Resultados tipados de cada comando
`medicion.py`	`emisor()` y `receptor()` para medir tasa efectiva y BER; `guardar_csv()`
`simulador.py`	`MedioSimulado(ber)` y `FaroSimulado`: imitan el firmware para probar sin hardware
`faro_gpio.py`	`FaroGPIO`: el Faro manejado por una Raspberry Pi (hereda los comandos de `FaroSimulado` y cambia el medio por luz real con pigpio); `DecodificadorManchester` (traducción del del firmware), `pulsos_manchester`, CSMA/CA, ACK, duplicados, pánico por botón y memoria en archivo
`scapy_faro.FaroTrama`	Capa Scapy de la trama, con CRC calculado en `post_build`

4.3 Nodo de la mesh (`nodo/faronet/`)

Módulo	Estructuras y funciones
`celda.py`	`Celda` (512 B; `a_bytes`, `desde_bytes`), comandos `HOLA`, `RUTAS`, `DIRECTORIO`, `CREAR`, `CREADA`, `RELE_IDA`, `RELE_VUELTA`, `DESTRUIR`, `ENTREGA`
`cripto.py`	`Identidad` (semilla, firma, abrir sobre, `borrar`), `cerrar_sobre`, `HandshakeCliente` y `handshake_rele` (ntor), `CapaOnion` (`cifrar`/`descifrar` por sentido)
`onion.py`	`envolver`, `desenvolver`, `quitar_capa`, `agregar_capa`, `originar`; constantes `SOBRE = 391`, `MIN_SALTOS = 3`, `MAX_SALTOS = 5`
`ruteo.py`	`TablaRutas` (`procesar_anuncio`, `siguiente`, `anuncio_para` con horizonte dividido, `enlace_caido`, `purgar`)
`directorio.py`	`Entrada` firmada (`publicar`, `desde_bytes` con verificación) y `Directorio` (`agregar`, `buscar`, `purgar`, `borrar`)
`enlaces.py`	`Enlace` abstracto; `EnlaceTCP`, `ServidorTCP`, `EnlaceFaro` (fragmentación en 5 tramas y reensamblado) y `EnlaceMemoria` para pruebas
`mensaje.py`	`Mensaje` privado (`sellar`, `abrir`, `vencido`), `MensajePublico`, `Buzon` con TTL
`nodo.py`	`Nodo` (ciclo de vida, `enviar_celda`, `construir_circuito`, `enviar_mensaje`, `crear_tema_privado`, `enviar_tema`, `publicar`, `panico`), `Tramo` (estado de un relé: solo anterior y siguiente) y `Circuito` (lado cliente)
`cli.py`	Cliente de terminal con parámetros por línea de comandos

4.4 Gateway (`gateway/faronet_gateway/`)

Elemento	Descripción
`Puente` (`puente.py`)	Se engancha al nodo gateway: publica los `MensajePublico` que llegan como salida de circuito y sube el directorio cada 5 min
`ClienteEdge` (`edge.py`)	Cliente HTTPS firmado del Worker: `publicar`, `subir_directorio`, `directorio`, `mensajes`
`id_buzon`, `sellar_buzon`, `abrir_buzon`, `depositar`, `retirar`	Buzón muerto: identificador y llave derivados del secreto compartido

4.5 Worker (`worker/src/`)

Archivo	Contenido
`index.ts`	Enrutador HTTP: páginas, API pública, buzón y rutas firmadas del gateway
`seguridad.ts`	`autenticarGateway` (firma, ventana de 60 s, anti-replay) y `permitir` (rate limiting con hash)
`sala.ts`	Durable Object `Sala`: WebSockets hibernables, historial en SQLite, `feedAtom`
`central.ts`	Durable Object `Central`: directorio verificado, salas activas y último tiempo por gateway
`buzon.ts`	Durable Object `Buzon`: blobs de 1024 B con TTL y lectura única
`paginas.ts`	HTML de inicio y de sala con CSP por nonce
`util.ts`	hex/base64, `sha256`, `verificarEd25519` (WebCrypto), respuestas JSON

5. Instrucciones para ejecutar

5.1 Preparar el Faro

Con una **Raspberry Pi** (lo que se usa en los Faros):

```
sudo apt install pigpio python3-pigpio
sudo systemctl enable --now pigpiod             # demonio de tiempos precisos
pip install -e "adaptador[gpio]" -e nodo -e gateway
faro -p gpio info                               # pines por defecto: gpio:17,27,22,5,6,13
```

Las conexiones están en la sección 3.1. El puerto `gpio` acepta otros pines (`gpio:EMISOR,RECEPTOR,BOTON,LED_TX,LED_RX,LED_ESTADO`) y un pigpiod remoto (`gpio@raspberry.local`).

Alternativa con **Arduino** (firmware de `firmware/faro/`):

```
arduino-cli core install arduino:avr
# Arduino Uno
arduino-cli compile --fqbn arduino:avr:uno firmware/faro
arduino-cli upload  --fqbn arduino:avr:uno -p /dev/ttyACM0 firmware/faro   # clones CH340: /dev/ttyUSB0
# Arduino Nano (clon CH340)
arduino-cli compile --fqbn arduino:avr:nano firmware/faro
arduino-cli upload  --fqbn arduino:avr:nano:cpu=atmega328old -p /dev/ttyUSB0 firmware/faro
```

Si no hay permiso sobre el puerto: `sudo usermod -aG dialout $USER` y volver a iniciar sesión. Pruebas del núcleo en la PC:

```
cd firmware/faro/test
g++ -std=c++11 -Wall -I.. test_core.cpp ../faro_core.cpp -o test_core && ./test_core
```

5.2 Instalar la biblioteca, el nodo y el gateway

```
python3 -m venv .venv && source .venv/bin/activate
pip install -e adaptador -e nodo -e gateway      # desde la raíz del repositorio
```

5.3 Probar el Faro

Con la Raspberry el puerto es `gpio`; con Arduino, el puerto serial:

```
faro -p gpio info
faro -p gpio enviar a1:b2:c3:d4 "hola"
faro -p /dev/ttyACM0 info
faro -p /dev/ttyACM0 sondear --continuo           # A0 y comparador
faro -p /dev/ttyACM0 enviar a1:b2:c3:d4 "hola"
faro -p /dev/ttyACM0 escuchar
faro -p /dev/ttyACM0 direccion --rotar
faro -p /dev/ttyACM0 configurar --bps 4000        # igual en los dos Faros

# Tasa efectiva y BER (una distancia por corrida)
faro -p /dev/ttyACM0 medir receptor --tramas 200 --tam 114 --distancia 1m --csv hardware/mediciones.csv
faro -p /dev/ttyUSB1 medir emisor   --tramas 200 --tam 114
```

Solo un programa a la vez puede usar el puerto serial, y cada vez que se abre el puerto el Uno se reinicia.

5.4 Ejecutar los nodos

```
# Nodo con identidad en archivo, escuchando y conectado a un vecino
faronet --llave ana.key --nombre ana --escuchar 0.0.0.0:9101 --vecino 192.168.1.20:9101

# Nodo con Faro: la identidad vive en la memoria del Faro y el pánico la borra
faronet --faro gpio --nombre puente-sur --escuchar 0.0.0.0:9101          # Raspberry
faronet --faro /dev/ttyACM0 --nombre puente-sur --escuchar 0.0.0.0:9101  # Arduino

# Gateway conectado al Worker
faronet --llave gw.key --nombre gw-norte --gateway \
        --edge https://faronet.faronet-worker.workers.dev --escuchar 0.0.0.0:9101

# Relé sin consola
faronet --llave rele.key --escuchar 0.0.0.0:9101 --vecino 10.0.0.5:9101 --sin-consola
```

Comandos de la consola: `nodos`, `rutas`, `circuitos`, `enviar <nombre|id> <texto>`, `publicar <tema> <texto>`, `tema crear|enviar|ver`, `buzon`, `panico`, `salir`. Demo local con 5 relés: `nodo/demo_local.sh`.

5.5 Desplegar el Worker

```
cd worker
npm install
npx wrangler login                                # una vez
faronet-edge llave --llave ~/llaves/gateway.key   # llave pública para GATEWAYS en wrangler.toml
npx wrangler deploy
```

Desarrollo local sin cuenta: `npx wrangler dev --var GATEWAYS:<llave_publica_hex>`.

5.6 Usar el edge desde la terminal

```
faronet-edge --url https://faronet.faronet-worker.workers.dev directorio
S=$(faronet-edge buzon secreto)
faronet-edge --url https://faronet.faronet-worker.workers.dev buzon depositar "$S" "nos vemos en el faro norte"
faronet-edge --url https://faronet.faronet-worker.workers.dev buzon retirar "$S"
```

Salas en el navegador: `https://faronet.faronet-worker.workers.dev/salas/general` y su feed en `/salas/general/feed.xml`.

5.7 Pruebas automáticas

```
cd adaptador && python -m unittest -v     # 22 pruebas (1 se salta sin g++/Scapy)
cd nodo      && python -m unittest -v     # 28 pruebas
cd gateway   && python -m unittest -v     # integración contra wrangler dev
```

6. Actividades realizadas por estudiante

[PENDIENTE] Cada integrante debe completar las horas a partir de su bitácora. Las actividades y fechas de abajo salen del historial de Git y de las sesiones de trabajo registradas; las horas están en blanco a propósito.

David Zamora

Fecha	Actividad	Horas
2026-09-30	Documento de kick-off y estructura del repositorio	
2026-10-15 a 2026-11-01	Revisión y fusión de los PR #1 a #8	
2026-10-26	Estudio de la guía de armado del Faro en protoboard	
2026-10-31	Armado del Faro en protoboard (Uno R3), pruebas 2 a 7 del receptor y emisor	
2026-10-31	Preparación de la Raspberry Pi 2 como nodo `faronet-sur`	
2026-11-01	Soldadura del Faro 2 en placa perforada: etapa 3A (rieles de 5 V y GND) y pruebas de corto	
2026-11-01 a 2026-11-02	Soldadura: etapas 3B (divisor de umbral) y 3C (receptor)	
	…	
	Total	

Diego Zamora

Fecha	Actividad	Horas
2026-10-15	Firmware del Faro y biblioteca DispositivoLuzAdaptador (Capa 1), CI	
2026-10-17	Mesh FaroNet: circuitos onion, ruteo dinámico, directorio firmado	
2026-10-17	Worker en el edge: salas, feeds, buzón muerto, directorio	
2026-10-17	Puente mesh → edge en el gateway	

Fecha	Actividad	Horas
2026-10-20	Faro en protoboard y calibración del receptor (`calibrar_receptor.py`)	
2026-11-01	Despliegue del Worker en Cloudflare y prueba de punta a punta	
2026-11-01	RFC de FaroNet y temas privados	
	…	
	Total	

7. Autoevaluación

7.1 Estado final

Componente	Estado al 2 de noviembre de 2026
Faro por Raspberry Pi (GPIO)	Completo; 8 pruebas con una Raspberry falsa; probado en la Raspberry Pi 2 (114 B en 0,53 s a 2000 bit/s)
Firmware Arduino (alternativa)	Completo; compila para Nano y Uno; núcleo probado en la PC
Biblioteca DispositivoLuzAdaptador	Completa, con simulador, Faro por GPIO, Scapy y comandos de medición; 30 pruebas pasan
Faro en protoboard	Funciona por separado en ambos Faros (recepción con LED propio y hoja / frente a frente)
Faro soldado	Faro 1 soldado y probado por partes (faltan cables de J1); Faro 2 sin el emisor todavía (fotos en la sección 3.1)
Prueba Faro contra Faro	**No realizada**
Tasa efectiva y BER a 3 distancias	**No realizada**. El procedimiento (Apéndice B del RFC) y la herramienta (`faro medir`) están listos, pero no hay resultados
Esquemático KiCad, bitácora de multímetro	**[PENDIENTE]** (esquemático y netlist en Markdown sí existen; las fotos ya están)
Mesh: ruteo, directorio, onion, mensajes privados, temas privados	Completa; 28 pruebas pasan (incluye red de 6 nodos y red que cruza dos Faros simulados)
Autenticación de `HOLA` y tráfico de relleno	**[PENDIENTE]**
Gateway y Worker	Completos y desplegados en https://faronet.faronet-worker.workers.dev
RFC	Completo (`docs/rfc-faronet.txt`); el Apéndice B indica que las mediciones no se realizaron

7.2 Problemas encontrados y limitaciones

- **No se pudieron hacer las pruebas Faro contra Faro ni las mediciones de tasa efectiva y BER a tres distancias** que pide el enunciado. Cada Faro se probó solo (emisión y recepción con su propio LED y una hoja, o frente a frente a 1 cm), pero no se llegó a comunicar dos Faros entre sí. Por eso no hay datos de velocidad ni de BER, el enlace por luz entre comunidades solo se probó con Faros simulados (pruebas de `nodo/`) y no se compite por el extra de mayor tasa de transferencia.

- Se reemplazó el Arduino por una Raspberry Pi por Faro. Eso obligó a reimplementar la capa 1 en la Raspberry (`faro_gpio.py`) y a bajar a 3,3 V la salida del comparador con un divisor.
- El trimpot del kit no variaba el voltaje: se reemplazó por un divisor fijo.
- El sensor del Faro de David es débil (probablemente un fotodiodo y no un fototransistor): el margen entre luz y oscuridad es justo para 1–2 m.
- Abrir el puerto serial reinicia el Uno y solo un programa puede usarlo a la vez; cambiar piezas con el USB conectado llegó a tumbar la Raspberry Pi.
- Faltaban valores de resistencia (1 MΩ, 4,7 kΩ, 10 kΩ) y se usaron equivalentes o combinaciones en paralelo.
- La placa perforada usada tiene la numeración de columnas invertida entre la cara de cobre y la de componentes, lo que obligó a rehacer los mapas.
- Limitaciones de diseño: enlaces entre vecinos sin cifrar, `HOLA` sin autenticar, sin tráfico de cobertura, sin defensa contra Sybil (sección 3.6).

7.3 Reporte de commits

Generado con `git log --date=short --format='%h %ad %an %s'` sobre `main`:

Commit	Fecha	Autor	Descripción
`b1d3ada`	2026-11-01	DavidZamoraB	Merge PR #8 (temas privados)
`21ae6d2`	2026-11-01	DavidZamoraB	Merge PR #7 (RFC)
`27757fc`	2026-11-01	Diego Zamora	RFC: sección 5.3 Temas privados
`1a20a88`	2026-11-01	Diego Zamora	Temas privados: residen únicamente dentro de FaroNet
`679f5ed`	2026-11-01	Diego Zamora	RFC de FaroNet: Capa 1, mesh anónima y edge
`5ccc60f`	2026-11-01	DavidZamoraB	Merge PR #6 (despliegue del Worker)
`a68638c`	2026-11-01	Diego Zamora	Worker desplegado en Cloudflare
`f5fee32`	2026-10-31	DavidZamoraB	Merge PR #5 (hardware del receptor)
`76f0070`	2026-10-24	Claude	Hardware: receptor probado por Diego
`14363c7`	2026-10-17	DavidZamoraB	Merge PR #4 (puente edge)

Commit	Fecha	Autor	Descripción
`337acee`	2026-10-17	Diego Zamora	Puente mesh → edge
`60b89eb`	2026-10-17	DavidZamoraB	Merge PR #2 (mesh onion)
`f78d7ff`	2026-10-17	Diego Zamora	Worker en el edge (Capa 7)
`e0deafa`	2026-10-17	Diego Zamora	Mesh FaroNet (Capas 2, 3 y 7)
`718f878`	2026-10-15	Diego Zamora Bustos	Merge PR #1 (Capa 1)
`2af63c3`	2026-10-15	Diego Zamora	CI: pruebas y compilación del firmware
`07cc883`	2026-10-15	Diego Zamora	Faro: hardware, firmware y biblioteca (Capa 1)
`8dfa32a`	2026-09-30	DavidZamoraB	docs: actualizar documento de kick-off
`569ea44`	2026-10-01	Equipo FaroNet	Estructura inicial del repositorio

Antes de entregar se debe regenerar esta tabla con el historial final.

7.4 Calificación según la rúbrica

[PENDIENTE] Cada ítem de 0 a 10, a llenar por el equipo.

Rubro	Peso	Nota (0–10)
Hardware: Faro soldado (esquemático, BOM, soldadura, pruebas)	25 %	
Capa 1: protocolo físico y DispositivoLuzAdaptador	10 %	
Capas 2 y 3: mesh anónima y directorio de nodos	15 %	
Privacidad: modelo de amenazas, cifrado, padding, pánico	10 %	
Capa 7: aplicación anónima a través de FaroNet	5 %	
Capa 7: aplicación pública en el edge	25 %	
RFC	5 %	
Documentación	5 %	
Kick-off	5 % extra	
Extra: mayor tasa de transferencia	10 %	

7.5 Autoevaluación de aprendizaje

Escala [1] Muy malo, [2] Malo, [3] Regular, [4] Bueno, [5] Muy bueno.

Rubro	David	Diego
Aprendizaje de soldadura y hardware	5	5
Aprendizaje de la capa física	5	5
Aprendizaje de enrutamiento en redes mesh	5	5
Aprendizaje de privacidad y anonimato	5	5
Aprendizaje de serverless en el edge	5	5
Aprendizaje de redacción de RFCs	5	5

8. Lecciones aprendidas

Para quien lleve este curso en el futuro:

1. **Empiecen por el hardware.** El software se puede escribir y probar con un simulador, pero el hardware depende de comprar piezas, de que las piezas sirvan y de aprender a soldar. Fue el cuello de botella.
2. **Compren repuestos y verifiquen el sensor.** Un “fototransistor” de tienda puede ser un fotodiodo con mucha menos corriente. Prueben cada pieza en protoboard antes de soldarla y compren con filtro de luz diurna.
3. **Prefieran valores fijos a trimpots baratos.** Un divisor de dos resistencias es más estable y más fácil de reproducir.
4. **Midan antes de energizar.** La prueba de continuidad entre +5 V y GND evita quemar el Arduino o el puerto USB de la computadora. No cambien piezas con el USB conectado.
5. **Dibujen la placa por la cara en la que van a soldar.** La placa perforada se ve en espejo desde el cobre; un mapa con la orientación equivocada cuesta horas.
6. **Hagan el protocolo independiente del hardware.** Separar el núcleo del firmware (`faro_core`) permitió probarlo en la PC con desfase de reloj y ruido simulados.
7. **Tamaño fijo en todo.** Celdas, sobres y blobs de tamaño fijo resuelven el padding de una vez y simplifican el análisis de privacidad.
8. **No inventen criptografía.** Con libsodium y WebCrypto basta; el trabajo real es el diseño del protocolo y el modelo de amenazas.
9. **Ramas, pull requests y CI desde el día uno.** Cada PR corrió las pruebas y compiló el firmware; `main` nunca quedó roto.
10. **El plan gratuito de Cloudflare alcanza**, pero solo con Durable Objects SQLite; revisen las limitaciones del plan antes de diseñar.

9. Bibliografía

- Bradner, S. *Key words for use in RFCs to Indicate Requirement Levels*. RFC 2119, 1997.
- Leiba, B. *Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words*. RFC 8174, 2017.
- Flanagan, H. y Ginoza, S. *RFC Style Guide*. RFC 7322, 2014.
- Cooper, A. et al. *Privacy Considerations for Internet Protocols*. RFC 6973, 2013.
- Fielding, R., Nottingham, M. y Reschke, J. *HTTP Semantics*. RFC 9110, 2022.
- Hedrick, C. *Routing Information Protocol*. RFC 1058, 1988.
- Josefsson, S. y Liusvaara, I. *Edwards-Curve Digital Signature Algorithm (EdDSA)*. RFC 8032, 2017.
- Langley, A., Hamburg, M. y Turner, S. *Elliptic Curves for Security*. RFC 7748, 2016.
- Saarinen, M-J. y Aumasson, J-P. *The BLAKE2 Cryptographic Hash and MAC*. RFC 7693, 2015.
- Nottingham, M. y Sayre, R. *The Atom Syndication Format*. RFC 4287, 2005.
- Arciszewski, S. *XChaCha: eXtended-nonce ChaCha and AEAD_XChaCha20_Poly1305*. draft-irtf-cfrg-xchacha, 2020.
- Dingledine, R., Mathewson, N. y Syverson, P. *Tor: The Second-Generation Onion Router*. USENIX Security, 2004.
- Goldberg, I., Stebila, D. y Ustaoglu, B. *Anonymity and one-way authentication in key exchange protocols*. Designs, Codes and Cryptography, 2013.
- Cheshire, S. y Baker, M. *Consistent Overhead Byte Stuffing*. IEEE/ACM Transactions on Networking, 1999.
- IEEE 802.3. *Codificación Manchester* (cláusula 7, 10BASE).
- libsodium. *Documentation*. https://doc.libsodium.org
- PyNaCl. *Documentation*. https://pynacl.readthedocs.io
- Cloudflare. *Workers, Durable Objects y Rate Limiting*. https://developers.cloudflare.com/workers/
- Arduino. *Language Reference* y *arduino-cli*. https://docs.arduino.cc
- Texas Instruments. *LM393 Dual Comparator Datasheet*.
- Moraga, K. *Proyecto 1: FaroNet*. Enunciado del curso Redes de Computadoras, ITCR, 2026.

10. Documentación del código fuente

El código está documentado en el propio repositorio:

- Cada carpeta tiene un `README.md` con propósito, instalación, uso y diseño (`firmware/`, `adaptador/`, `nodo/`, `gateway/`, `worker/`, `hardware/`).
- Cada módulo Python empieza con un docstring que explica el formato que implementa (por ejemplo `nodo/faronet/onion.py` explica la aritmética de las capas y `nodo/faronet/mensaje.py` el formato del sobre).
- El firmware comenta cada bloque (`faro_core.h` documenta el decodificador Manchester y sus estados).
- El Worker comenta en cada archivo las decisiones de seguridad y no registro.
- `docs/capa1-protocolo.md` y el RFC especifican los formatos byte a byte.

Anexo: RFC de FaroNet

El RFC completo (`docs/rfc-faronet.txt`) se adjunta a continuación en su formato ASCII original.

Proyecto 1 - Redes de Computadoras
Instituto Tecnologico de Costa Rica
Request for Comments: FARONET-1
Category: Experimental

D. Zamora
D. Zamora
ITCR

Noviembre 2026

FaroNet: Enlace Optico (Faro), Red Mesh Anonima y Salida Serverless en el Edge

Resumen

Este documento especifica FaroNet, una red de mensajeria construida desde el medio fisico hasta la aplicacion. Define (1) la Capa 1 del "Faro", un dispositivo que transmite tramas de hasta 128 bytes con luz infrarroja, junto con el protocolo serial entre el Faro y la computadora del nodo; (2) la mesh FaroNet, que transporta celdas de tamano fijo por Ethernet, WiFi o el Faro, con enrutamiento por vector de distancias, un directorio de nodos firmado y circuitos onion de 3 a 5 saltos, de modo que ningun nodo intermedio conoce a la vez el origen y el destino de un mensaje; y (3) la salida a Internet a traves de un gateway que publica solo los mensajes publicos en una aplicacion serverless (Cloudflare Workers).

Estado de este memo

Este documento es un entregable academico del curso Redes de Computadoras (ITCR, II semestre 2026). No es un estandar de Internet ni fue revisado por el IETF. Sigue la guia de estilo de RFC 7322 y usa ASCII en todo el texto, incluidos los diagramas. Por esa razon el texto en espanol se escribe sin tildes.

Tabla de contenido

1. Introduccion
   1.1. Lenguaje de requisitos
   1.2. Terminologia
   1.3. Arquitectura
2. Capa 1: el Faro
   2.1. Medio y codificacion de linea
   2.2. Sincronizacion
   2.3. Formato de la trama
   2.4. Direcciones fisicas
   2.5. Acceso al medio
   2.6. Tramas armadas fuera del Faro
3. Interfaz nodo-Faro (DispositivoLuzAdaptador)
4. FaroNet: capas 2 y 3
   4.1. Identidad de los nodos
   4.2. Celdas
   4.3. Enlaces
   4.4. Enrutamiento dinamico
   4.5. Directorio de nodos
   4.6. Circuitos virtuales
   4.7. Capas de la cebolla
   4.8. Mensajes de rele
5. Capa 7 dentro de FaroNet
   5.1. Mensajes privados
   5.2. Mensajes publicos
   5.3. Temas privados
6. Capa 7 en el edge
   6.1. Relay del gateway
   6.2. Salas publicas y feeds
   6.3. Buzon muerto
   6.4. Directorio publico
   6.5. Limite de peticiones y politica de no registro
7. Versionado
8. Modelo de amenazas

9. Consideraciones de privacidad
10. Consideraciones de seguridad
11. Consideraciones de IANA
12. Referencias
12.1. Referencias normativas
12.2. Referencias informativas
Apendice A. Constantes
Apendice B. Medicion de tasa y BER
Direcciones de los autores

1. Introduccion

El cifrado protege el contenido de un mensaje, pero no los metadatos: quien habla con quien, cuando y con que frecuencia. FaroNet busca proteger tambien esos metadatos.

La red une "comunidades" separadas. Dentro de una comunidad los nodos se conectan por TCP/IP (Ethernet o WiFi). Entre comunidades el unico canal es un par de Faros que se comunican con luz. Un gateway conecta la red con Internet, pero solo deja salir mensajes que el remitente marco como publicos, y lo hace sin saber quien los origino.

El codigo de referencia esta en https://github.com/DavidZamoraB/faronet: el firmware en "firmware/", la biblioteca DispositivoLuzAdaptador en "adaptador/", los nodos en "nodo/", el gateway en "gateway/" y el Worker en "worker/".

1.1. Lenguaje de requisitos

Las palabras clave "DEBE", "NO DEBE", "REQUERIDO", "DEBERIA", "NO DEBERIA", "RECOMENDADO", "PUEDE" y "OPCIONAL" se interpretan como "MUST", "MUST NOT", "REQUIRED", "SHOULD", "SHOULD NOT", "RECOMMENDED", "MAY" y "OPTIONAL" de BCP 14 [RFC2119] [RFC8174] cuando aparecen en mayusculas.

1.2. Terminologia

Faro: Dispositivo de transmision por luz: un emisor IR de 940 nm, un receptor con fototransistor y comparador, LEDs de estado y un boton de panico. Lo controla una Raspberry Pi por sus pines GPIO (implementacion de referencia) o un microcontrolador (Arduino) conectado por USB al nodo.

Nodo: Programa FaroNet que corre en una computadora. Todo nodo es rele; algunos ademas son gateway.

Celda: Unidad de las capas 2 y 3 de FaroNet. Siempre mide 512 bytes.

Circuito: Camino virtual negociado de 3 a 5 reles que un cliente usa para enviar mensajes.

Salida: Ultimo rele de un circuito.

Gateway: Nodo que, ademas de ser rele, publica mensajes publicos en el edge.

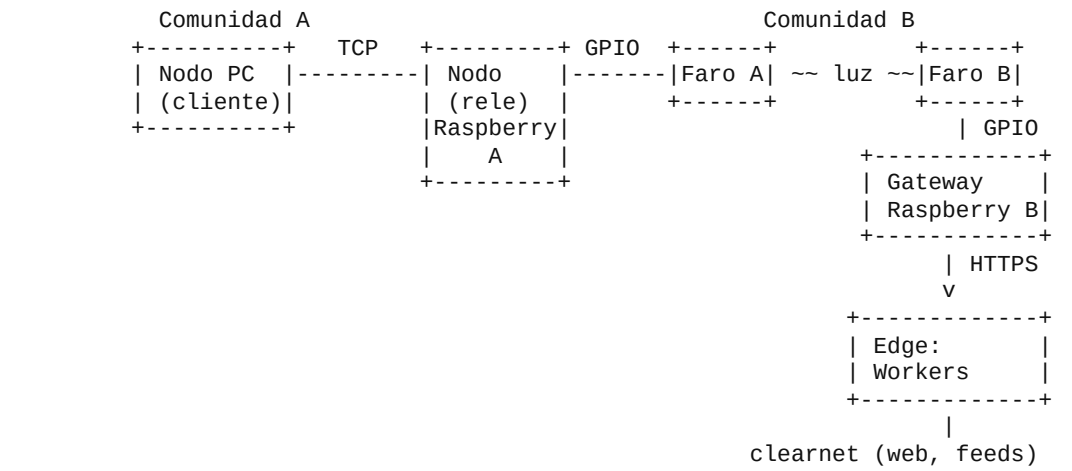
Edge: Aplicacion serverless (Cloudflare Workers) que expone salas, feeds, buzon muerto y directorio a Internet ("clearnet").

ID de nodo: 8 bytes derivados de la llave publica de firma del nodo (Seccion 4.1).

Direccion fisica: 4 bytes que identifican a un Faro en el medio

optico (Seccion 2.4). No tiene relacion con el ID de nodo.

1.3. Arquitectura



Capas implementadas:

Capa	Funcion	Seccion
7 (edge)	Salas, feeds, buzon muerto, directorio	6
7 (mesh)	Mensajes privados y publicos	5
onion	Circuitos de 3 a 5 saltos, una capa AEAD por salto	4.6-4.8
3	Celdas con destino de nodo, vector de distancias, directorio firmado	4.2,4.4
2	Enlaces TCP y Faro (fragmentacion)	4.3
1	Faro: Manchester, trama, CRC, CSMA/CA	2, 3

2. Capa 1: el Faro

2.1. Medio y codificacion de linea

El medio es luz infrarroja de 940 nm, modulada por encendido y apagado (OOK) con codificacion Manchester segun la convencion de IEEE 802.3: el bit 0 es una transicion de encendido a apagado y el bit 1 de apagado a encendido, a la mitad del tiempo de bit. En reposo el emisor DEBE estar apagado.

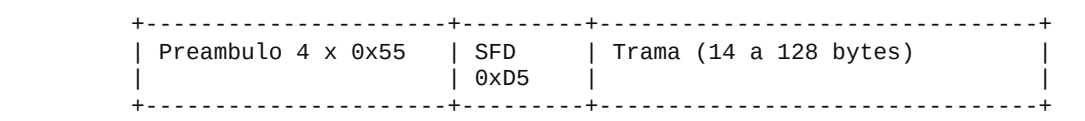
La velocidad es configurable de 500 a 20000 bit/s; por defecto es 2000 bit/s. Los dos extremos de un enlace DEBEN usar la misma velocidad.

Se eligio Manchester sobre OOK simple porque:

- Es autosincronizante: siempre hay un flanco a mitad de bit, asi que el receptor corrige el desfase entre los relojes de los dos Faros. El decodificador tolera un error de +/-50% en cada intervalo.
- Tiene balance de continua: la mitad del tiempo hay luz, lo que permite fijar el umbral del comparador una sola vez.
- Detecta errores de linea: un intervalo que no mide ni medio bit ni un bit completo es imposible, y la trama se descarta antes de llegar al CRC.

El costo es duplicar el ancho de banda (2 baudios por bit).

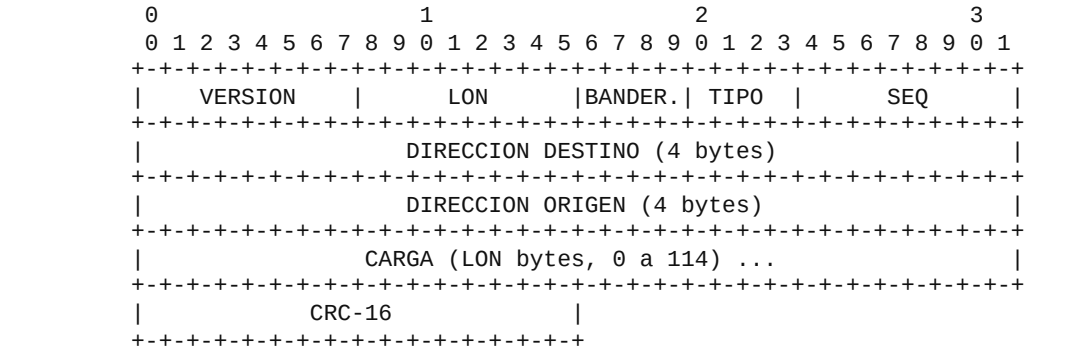
2.2. Sincronizacion



El preambulo 0x55 produce en Manchester solo intervalos de un bit completo. Despues de 8 intervalos seguidos el receptor sabe que cada flanco cae a mitad de bit. El SFD rompe la alternancia: el receptor busca el patron de 16 bits 0x55D5. Si encuentra 0xAA2A (el patron negado), la senal llega invertida y el receptor DEBE invertir los bits siguientes, de modo que la polaridad del hardware no importa.

Los bytes se transmiten con el bit mas significativo primero. El preambulo y el SFD no cuentan dentro de los 128 bytes de la trama.

2.3. Formato de la trama



VERSION (1 byte): 0x01. Un receptor DEBE descartar las tramas de versiones que no conoce. En modo promiscuo PUEDE entregarlas al nodo.

LON (1 byte): Longitud de la CARGA. Es el campo aparte que define el tamano del unico campo dinamico de la trama.

BANDERAS (4 bits): El bit 0x10 del byte es PIDE_ACK. Los demas son reservados, DEBEN valer 0 al enviar y DEBEN ignorarse al recibir.

TIPO (4 bits): 0 = DATOS, 1 = ACK. Los demas valores quedan reservados.

SEQ (1 byte): Numero de secuencia. Un ACK repite el SEQ de la trama que confirma.

DESTINO (4 bytes): Direccion fisica destino. FF:FF:FF:FF es difusion.

ORIGEN (4 bytes): Direccion fisica de quien transmite.

CARGA (LON bytes): Datos de la capa superior, maximo 114 bytes.

CRC-16 (2 bytes): CRC-16/CCITT-FALSE (polinomio 0x1021, valor inicial 0xFFFF, sin reflejar), en orden de red, calculado sobre todos los bytes anteriores de la trama.

La trama mide 12 + LON + 2 bytes, entre 14 y 128.

2.4. Direcciones fisicas

Una direccion fisica mide 4 bytes y se guarda en la memoria no volatil del Faro (un archivo en la tarjeta SD de la Raspberry Pi, o la EEPROM del Arduino).
00:00:00:00 y FF:FF:FF:FF NO DEBEN usarse como direccion propia.

Las direcciones son rotables. El nodo DEBERIA generar una nueva con el generador criptografico del sistema operativo y fijarla en el Faro (comandos FIJAR_DIRECCION o ROTAR_DIRECCION, Seccion 3). La biblioteca ofrece rotacion periodica con +/-20% de variacion aleatoria en el intervalo, para que el momento del cambio no identifique al dispositivo.

Con 4 bytes hay 2^32 direcciones. En una red de decenas de Faros la probabilidad de choque entre direcciones aleatorias es del orden de n^2 / 2^33, despreciable, y cada trama ahorra 4 bytes frente a usar 6 como Ethernet.

2.5. Acceso al medio

El canal es semiduplex y un Faro no puede escuchar mientras transmite, asi que no hay deteccion de colisiones. Las colisiones se evitan con CSMA/CA:

1. Escucha de portadora: el canal esta ocupado si el decodificador esta dentro de una trama o si ve intervalos Manchester validos seguidos. La luz ambiental no forma esos intervalos.

2. Espera aleatoria de 0 a (ventana - 1) ranuras de 24 bits. La ventana empieza en 8 y se duplica en cada reintento, hasta 64.

3. DIFS: el canal DEBE estar libre durante 32 bits seguidos antes de transmitir. Si en 3 s nunca queda libre, el Faro reporta CANAL_OCUPADO.

4. ACK: una trama unicast con PIDE_ACK se confirma con una trama TIPO=ACK sin carga (14 bytes), enviada despues de un SIFS de 12 bits sin escuchar el canal. Como SIFS < DIFS, el ACK tiene prioridad sobre cualquier otra transmision.

5. Si el ACK no llega en SIFS + 48 bits + la duracion de un ACK, la trama se retransmite, hasta 3 veces por defecto (configurable de 0 a 7).

6. El receptor recuerda los ultimos 8 pares (ORIGEN, SEQ). Una retransmision repetida se vuelve a confirmar, pero NO DEBE entregarse dos veces al nodo.

Las tramas de difusion NO DEBEN pedir ACK.

2.6. Tramas armadas fuera del Faro

El nodo PUEDE entregar al Faro una trama completa, por ejemplo construida con Scapy (adaptador/dispositivo_luz_adaptador/scapy_faro.py), con el comando ENVIAR_CRUDA. El Faro solo verifica que la longitud sea coherente con LON y la transmite tal cual, sin recalcular el CRC. Esto permite probar versiones futuras del protocolo sin cambiar el firmware.

3. Interfaz nodo-Faro (DispositivoLuzAdaptador)

Cuando el Faro lo controla una Raspberry Pi, la Capa 1 corre en la misma Raspberry que el nodo (adaptador/dispositivo_luz_adaptador/faro_gpio.py) y este protocolo se usa dentro del proceso, sin USB. Los tiempos los maneja pigpio: cada trama sale como forma de onda por DMA y cada flanco del receptor llega con su marca de tiempo en

microsegundos, asi que la latencia del sistema operativo no afecta la decodificacion. La salida del comparador DEBE bajarse a 3,3 V (por ejemplo con un divisor resistivo) antes de entrar a la Raspberry.

Con un Arduino, el Faro se comunica con el nodo por USB-serial a 115200 baudios, 8N1. Cada paquete es:

```
COBS( TIPO (1) | DATOS | CRC-16 (2) ) || 0x00
```

COBS elimina los bytes 0x00 del contenido, asi que 0x00 marca sin ambiguedad el fin del paquete, y el receptor se resincroniza despues de cualquier basura en la linea. El nodo envia un comando a la vez y espera la respuesta. Los eventos pueden llegar en cualquier momento.

Cod.	Comando	Datos	Respuesta
0x01	PING	-	0x81 PONG
0x02	INFO	-	0x82 INFO
0x03	ENVIAR	destino(4) banderas carga	0x83 RESULTADO_TX
0x04	ENVIAR_CRUDA	trama completa	0x83
0x05	FIJAR_DIRECCION	direccion(4)	0x85
0x06	ROTAR_DIRECCION	-	0x85
0x07	CONFIGURAR	bps(2) opciones reintentos	0x82 INFO
0x08	ESTADISTICAS	-	0x88
0x09	REINICIAR_EST.	-	0x88
0x0A	GUARDAR_SECRETO	32 bytes	0x8A
0x0B	LEER_SECRETO	-	0x8B
0x0C	PANICO	-	evento 0x8C
0x0D	SONDEAR	-	0x8D
0x0E	EMISOR_FIJO	0 o 1	0x8D

Eventos: 0x90 TRAMA_RECIBIDA (crc_ok, trama) y 0x8C PANICO (nueva direccion). Errores: 0xFF seguido de un codigo (1 comando desconocido, 2 longitud, 3 CRC del enlace serial, 4 parametro, 5 sin secreto).

Estados de transmision: 0 OK, 1 SIN_ACK, 2 CANAL_OCUPADO, 3 TRAMA_INVALIDA, 4 ABORTADO_PANICO. Opciones: bit 0 = promiscuo, bit 1 = entregar tramas con CRC malo (para medir el BER).

PONG incluye la version mayor y menor del firmware, la version del protocolo de Capa 1 y el tamano maximo de trama, de modo que el nodo DEBE verificar la compatibilidad antes de usar el Faro.

GUARDAR_SECRETO almacena en la memoria no volatil del Faro la semilla de identidad del nodo (Seccion 4.1). El boton de panico la borra (Seccion 9).

4. FaroNet: capas 2 y 3

4.1. Identidad de los nodos

Cada nodo tiene una semilla secreta de 32 bytes. De ella derivan:

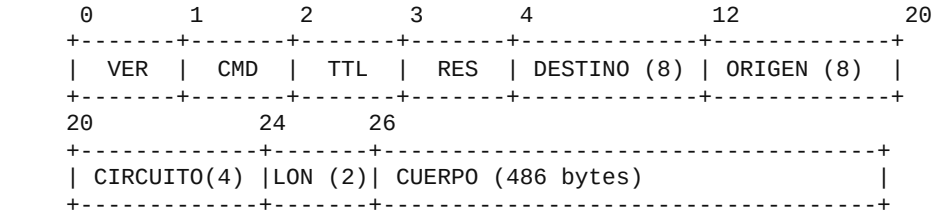
- Una llave de firma Ed25519, que firma las entradas del directorio.
- Una llave "onion" X25519, con la que se negocian los circuitos y se reciben los mensajes privados.

El ID de nodo es un hash BLAKE2b de 8 bytes de la llave publica de firma. La semilla PUEDE guardarse en un archivo (opcion --llave) o en la memoria no volatil del Faro, donde el boton de panico la borra.

Toda la criptografia usa libsodium a traves de PyNaCl en los nodos y WebCrypto en el Worker. FaroNet NO DEFINE primitivas propias.

4.2. Celdas

Todas las celdas miden exactamente 512 bytes, para no filtrar la longitud de los mensajes.



VER: 0x01.

CMD: Comando (tabla abajo).

TTL: Saltos restantes, 16 al crear la celda. Un nodo que reenvia una celda DEBE decrementarlo y DEBE descartarla si llega a 0.

RES: Reservado, 0.

DESTINO, ORIGEN: IDs de nodo de un solo tramo del circuito (por ejemplo, del rele 1 al rele 2), nunca del mensaje de extremo a extremo. Las celdas que solo cruzan un enlace usan DESTINO en ceros.

CIRCUITO: Identificador del circuito en ese tramo, elegido al azar por quien crea el tramo.

LON: Cuantos bytes del CUERPO son significativos. El resto es relleno aleatorio.

CMD	Nombre	Uso
0x01	HOLA	Presenta el ID del vecino (un solo enlace)
0x02	RUTAS	Anuncio del vector de distancias
0x03	DIRECTORIO	Una entrada firmada del directorio
0x04	RELLENO	Reservado para trafico de cobertura
0x10	CREAR	Primer paso del handshake de un tramo
0x11	CREADA	Respuesta del handshake
0x12	RELE_IDA	Cuerpo onion del cliente hacia la salida
0x13	RELE_VUELTA	Cuerpo onion de la salida hacia el cliente
0x14	DESTRUIR	Cierra un circuito en un tramo
0x15	ENTREGA	La salida entrega un sobre al destinatario

4.3. Enlaces

Un enlace mueve celdas completas, en orden, a un vecino.

TCP (Ethernet, WiFi): Las celdas se escriben una tras otra en el flujo TCP. Como todas miden 512 bytes, no hace falta delimitarlas.
Costo del enlace: 1.

Faro: Una celda no cabe en una trama, asi que se fragmenta. Cada fragmento viaja en la CARGA de una trama:

```
+-----------+--------+-------+-------------------------+
| ID_CELDA  | INDICE | TOTAL | DATOS (hasta 112 bytes) |
| 1 byte    | 4 bits | 4 bits|                         |
+-----------+--------+-------+-------------------------+
```

Una celda ocupa 5 fragmentos. Los fragmentos van unicast con ACK a la ultima direccion fisica vista del otro Faro. Como las direcciones rotan, si no llega el ACK el fragmento se repite por difusion. Las celdas incompletas se descartan a los 30 s. Si una celda se pierde, los circuitos que la usaban se destruyen y se reconstruyen. Costo del enlace: 4.

Al abrir un enlace cada lado envia HOLA con su ID de nodo.

4.4. Enrutamiento dinamico

FaroNet usa un vector de distancias al estilo RIP. Cada nodo envia a sus vecinos, cada intervalo (10 s por defecto), celdas RUTAS con pares (ID de destino de 8 bytes, metrica de 1 byte), hasta 53 por celda. La metrica es la suma de los costos de los enlaces; 16 es infinito.

Se usa horizonte dividido con envenenamiento en reversa, y las rutas que no se refrescan en 3 intervalos expiran. Una celda con DESTINO distinto del propio ID DEBE reenviarse por el enlace de la ruta de menor metrica: "al siguiente dispositivo que esta mas cerca del destino".

4.5. Directorio de nodos

Cada nodo publica una entrada firmada:

```
+-----+----+-------------+-------------+-------+-----------+--------+
| VER | ID | LLAVE FIRMA | LLAVE ONION | ROLES | PUBLICADA | EXPIRA |
| 1   | 8  | 32          | 32          | 1     | 4         | 4      |
+-----+----+-------------+-------------+-------+-----------+--------+
+---------+--------------+------------+
| LON_NOM | NOMBRE       | FIRMA      |
| 1       | LON_NOM      | 64         |
+---------+--------------+------------+
```

ROLES: 0x01 rele, 0x02 gateway.

PUBLICADA, EXPIRA: Segundos desde 1970. Una entrada dura 10 minutos. Un nodo DEBE rechazar entradas que pidan mas de 1 hora de vigencia o que esten fuera de una tolerancia de reloj de 5 minutos.

NOMBRE: Seudonimo elegido por el nodo, hasta 32 bytes.

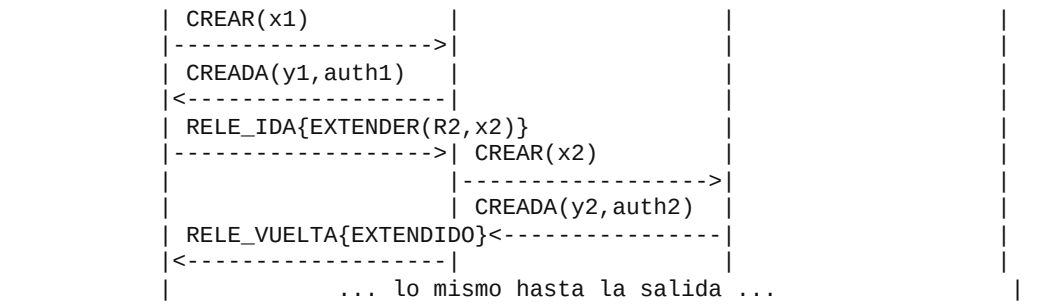
FIRMA: Ed25519 sobre todos los bytes anteriores. ID DEBE ser el ID derivado de LLAVE FIRMA.

Las entradas se difunden por inundacion (celdas DIRECTORIO) con supresion de duplicados, asi que cada nodo tiene su copia y no hay un servidor central que pueda mentir: una entrada alterada no verifica. Las entradas NO DEBEN contener direcciones IP ni fisicas: los nodos se alcanzan por ID usando las rutas de la mesh.

4.6. Circuitos virtuales

Un cliente elige de su directorio entre 3 y 5 reles al azar (3 por defecto); el ultimo es la salida. El circuito se negocia salto por

salto, de manera similar a Tor:



El handshake (tipo ntor) usa dos Diffie-Hellman X25519: efimera del cliente con efimera del rele (secreto hacia adelante) y efimera del cliente con la llave onion estatica del rele, publicada en el directorio (autentica al rele). Con BLAKE2b sobre los dos secretos y el transcrito (ID del rele, llave onion, x, y) se derivan una llave para cada sentido y un autenticador de 32 bytes. Si el autenticador no coincide, el cliente DEBE abortar el circuito.

Cada rele guarda solo un "tramo": el nodo anterior, el siguiente, los IDs de circuito de cada lado y sus llaves. Asi el primer rele conoce al cliente pero no el destino, la salida conoce el destino pero no al cliente, y el rele del medio no conoce a ninguno. Un tramo sin trafico se olvida a los 10 minutos.

4.7. Capas de la cebolla

El cuerpo de RELE_IDA y RELE_VUELTA se cifra con XChaCha20-Poly1305 (AEAD) una vez por salto. El nonce es un contador por sentido: los enlaces entregan en orden y sin duplicados, asi que no viaja en la celda. Si una celda se pierde, el siguiente descifrado falla y el circuito se destruye.

Cada capa agrega 16 bytes de etiqueta. Para que la celda siga midiendo lo mismo, cada salto quita su capa y completa con relleno aleatorio nuevo:

```
salto p recibe:  [ cifrado para p   : 486-(p-1)*16 ][ relleno ]
salto p envia:   [ cifrado para p+1 : 486-p*16     ][ relleno ]
```

El relleno se renueva en cada salto, de modo que un observador no puede seguir una celda comparando bytes.

4.8. Mensajes de rele

Texto en claro de una capa:

RECONOCIDO	COMANDO	LON (2)	DATOS	relleno
4 x 0x00	1		LON bytes	

Si al quitar su capa un salto ve RECONOCIDO en ceros, el mensaje es para el. Si no, es el cifrado del siguiente salto. (La probabilidad de que un cifrado empiece con 4 ceros es 2^-32; en ese caso el mensaje no se puede interpretar y el circuito se destruye.)

Cod.	Comando	Datos
0x01	EXTENDER	destino(8) | x(32) | posicion(1)
0x02	EXTENDIDO	y(32) | auth(32)

0x03	ENTREGAR	destinatario(8)	sobre(391)
0x04	ENTREGADO	estado(1): 0 OK, 1 sin ruta
0x05	ERROR	codigo(1)
0x06	PUBLICAR	mensaje publico (391)
0x07	PUBLICADO	estado(1): 0 OK, 1 la salida no es gateway, 2 el edge rechazo o no respondio

El sobre mide siempre 391 bytes: 486 - 5*16 (cinco capas como maximo) - 7 (cabecera) - 8 (destinatario).

5. Capa 7 dentro de FaroNet

5.1. Mensajes privados

El texto en claro de un mensaje privado siempre mide 343 bytes:

VER	TIPO	CREADO	TTL	LON_ALIAS	ALIAS
1	1	4	4	1	LON_ALIAS

LON	TEXTO	relleno (ceros hasta 343 bytes)
2	LON	

TIPO: 0x01 = privado; 0x02 = mensaje de un tema privado. Con TIPO 0x02, despues de ALIAS van LON_TEMA (1 byte) y TEMA (Seccion 5.3).

CREADO, TTL: Segundos desde 1970 y vida del mensaje en segundos.

Se cierra con SealedBox de libsodium (X25519 + XSalsa20-Poly1305) hacia la llave onion del destinatario, tomada del directorio. El sobre resultante mide 343 + 48 = 391 bytes.

El cliente lo envia con ENTREGAR por un circuito. La salida lo reenvia en una celda ENTREGA al destinatario por la ruta normal de la mesh. Ningun rele puede leerlo: solo el destinatario tiene la llave privada. ALIAS es un seudonimo opcional; FaroNet no agrega ninguna identidad del remitente.

Los mensajes privados residen unicamente dentro de FaroNet: un gateway NO DEBE publicarlos en el edge. El destinatario los guarda solo en memoria y los descarta al vencer su TTL (24 horas por defecto).

5.2. Mensajes publicos

Un mensaje publico tambien mide 391 bytes, rellenado con ceros:

VER	LON_TEMA	TEMA	LON_ALIAS	ALIAS	LON	TEXTO	relleno
1	1		1		2		ceros

TEMA: Minusculas, numeros y guiones, hasta 32 caracteres.

TEXTO: Hasta 500 caracteres (limite del edge).

El cliente construye un circuito cuya salida es un nodo con el rol gateway y le envia PUBLICAR. El gateway lo sube al edge (Seccion 6.1) y responde PUBLICADO. El gateway ve el texto, el tema y el alias, que de todas formas seran publicos, pero no sabe quien lo origino.

5.3. Temas privados

Un tema privado reside unicamente dentro de FaroNet. Quien lo crea guarda en memoria la lista de sus miembros (nombres o IDs del directorio). Cada mensaje del tema se envia a cada miembro como un mensaje privado (Seccion 5.1) con TIPO 0x02, por un circuito propio y sellado hacia la llave de ese miembro. El nombre del tema viaja dentro del sobre, asi que ni los reles ni el gateway lo ven, y el sobre mide los mismos 391 bytes.

Un nodo NO DEBE publicar con PUBLICAR un tema que tenga marcado como privado. La lista de miembros NO DEBE escribirse a disco, y el boton de panico la borra.

Costo: un tema con n miembros produce n circuitos, y cada miembro recibe el mensaje en un momento distinto. Un observador que vea varios destinos recibir mensajes en rafaga podria sospechar que son del mismo tema (ver Seccion 8).

6. Capa 7 en el edge

La aplicacion publica corre en Cloudflare Workers (plan gratuito) con Durable Objects. El despliegue es reproducible con "npx wrangler deploy" y el archivo worker/wrangler.toml del repositorio, que no contiene secretos.

Metodo	Ruta	Funcion
GET	/	Pagina con las salas activas
GET	/salas/:tema	Sala en vivo (WebSocket)
GET	/salas/:tema/ws	WebSocket de la sala
GET	/salas/:tema/feed.xml	Feed Atom [RFC4287]
GET	/salas/:tema/mensajes	Mensajes recientes (JSON)
GET	/api/salas	Salas activas
GET	/api/directorio	Replica del directorio
PUT	/api/buzon/:id?ttl=SEG	Deposita un blob
GET	/api/buzon/:id	Retira un blob (se borra)
POST	/api/gateway/publicar	Firmado: publica un mensaje
POST	/api/gateway/directorio	Firmado: sube el directorio

6.1. Relay del gateway

El gateway habla con el Worker por HTTPS. Cada peticion lleva:

```
X-Faronet-Gateway: llave publica Ed25519 del gateway (hex)
X-Faronet-Tiempo:  milisegundos desde 1970 (decimal)
X-Faronet-Firma:   firma Ed25519 (hex) de

   "faronet-edge-v1" LF METODO LF RUTA LF TIEMPO LF
   hex(SHA-256(cuerpo))
```

El Worker DEBE verificar que la llave este en la lista GATEWAYS, que TIEMPO este a menos de 60 s de su reloj, que la firma sea valida y que TIEMPO sea estrictamente mayor que el ultimo aceptado de ese gateway (guardado en el Durable Object Central). Asi una peticion capturada no se puede repetir (replay) ni alterar.

El gateway usa como llave de firma la misma identidad Ed25519 de su nodo FaroNet, y sube todas las entradas de su directorio cada 5 minutos.

6.2. Salas publicas y feeds

Cada tema es un Durable Object con almacenamiento SQLite. Los navegadores se conectan por WebSocket y reciben el historial y luego cada mensaje nuevo en tiempo real. Las salas son de solo lectura desde Internet: solo un gateway autorizado publica. Guardan como maximo 200 mensajes y 7 dias. Cada mensaje nuevo aparece tambien en el feed Atom de su tema.

6.3. Buzon muerto

Dos personas que comparten un secreto de 32 bytes derivan, fuera del Worker y con BLAKE2b con distinto contexto, un identificador opaco de buzon y una llave. El texto se cifra con SecretBox (XSalsa20-Poly1305) y se rellena a un blob de 1024 bytes.

El Worker solo ve el identificador y 1024 bytes cifrados. El TTL va de 1 minuto a 72 horas (24 horas por defecto). La lectura es unica y atomica: el blob se borra al ser leido, y dos lectores nunca reciben el mismo blob. Si el identificador ya esta ocupado, el deposito se rechaza.

6.4. Directorio publico

El Worker verifica la firma Ed25519 de cada entrada que sube un gateway y publica los bytes originales (en base64) junto con los campos decodificados, para que cualquiera pueda volver a verificar la firma. Las entradas vencidas se eliminan.

6.5. Limite de peticiones y politica de no registro

Cada cliente puede hacer 60 peticiones por minuto, con el limitador de Cloudflare (o un contador en memoria si la cuenta no lo ofrece). La clave del limite es SHA-256(fecha | clase | IP): la IP no se guarda.

El codigo del Worker NO DEBE persistir ni escribir en registros las cabeceras CF-Connecting-IP, User-Agent ni otras del visitante, y la observabilidad de Workers esta desactivada en wrangler.toml. Las paginas usan una politica de seguridad de contenido (CSP) estricta.

7. Versionado

Todos los formatos empiezan con un byte de version y todos los campos de largo variable tienen su largo en un campo aparte:

Formato	Version	Campo de largo
Trama del Faro	0x01	LON
Celda	0x01	LON
Entrada directorio	0x01	LON_NOMBRE
Mensaje privado	0x01	LON_ALIAS, LON_TEMA, LON
Mensaje publico	0x01	LON_TEMA, LON_ALIAS, LON
Mensaje de rele	-	LON
Firma del edge	"v1"	(contexto "faronet-edge-v1")

En toda version futura de la trama del Faro, VERSION y LON DEBEN seguir siendo los bytes 0 y 1, y la trama DEBE medir 12 + LON + 2 bytes. Asi cualquier receptor sabe donde termina una trama aunque no entienda su version. Un receptor DEBE descartar celdas, entradas y mensajes de versiones desconocidas.

8. Modelo de amenazas

Objetivo: ningun adversario individual DEBE poder saber a la vez quien envia un mensaje y a quien (o que el remitente es el autor de un mensaje publico).

Para cada adversario: que puede ver (V), que no debe ver (N) y la mitigacion (M).

Nodo malicioso de la mesh:
- V: su vecino anterior y siguiente, la hora y la cantidad de celdas.
- N: el origen y el destino a la vez; el contenido.
- M: circuitos de 3 a 5 saltos; celdas de 512 bytes; relleno aleatorio nuevo en cada salto; cifrado de extremo a extremo.

Observador del medio fisico (luz):
- V: que hay transmision, cuando, el tamano de cada trama y las direcciones fisicas actuales.
- N: el contenido; el seguimiento de un Faro a lo largo del tiempo.
- M: cifrado por capas y de extremo a extremo; rotacion de la direccion fisica con intervalo aleatorio.

Operador del gateway:
- V: los mensajes publicos que salen, su hora, tema y alias.
- N: el autor de los mensajes publicos; los mensajes privados.
- M: los privados nunca salen de FaroNet; el gateway es la salida de un circuito y no conoce al cliente.

Proveedor del edge (Cloudflare):
- V: la IP del gateway y de los navegadores, las horas, los tamanos y las rutas de las peticiones (incluido el identificador del buzon).
- N: el contenido del buzon; quien origino un mensaje dentro de FaroNet.
- M: blobs cifrados de tamano fijo con TTL; el Worker no guarda IP ni User-Agent; la clave del limite de peticiones es un hash.

Ladron del Faro:
- V: la memoria no volatil del Faro (EEPROM o tarjeta SD).
- N: las llaves y la identidad, si se activo el panico.
- M: boton de panico que borra el secreto y cambia la direccion.

Fuera de alcance:

- Un adversario global que observe al mismo tiempo la entrada y la salida de un circuito puede correlacionar por tiempo. FaroNet no agrega retardos ni trafico de cobertura (RELLENO esta reservado para eso).

- Varios reles maliciosos que colaboren y sean el primero y el ultimo salto de un mismo circuito.

- Ataques Sybil: cualquiera puede publicar muchas entradas en el directorio.

- Un adversario con acceso al puerto USB del Faro antes del panico.

9. Consideraciones de privacidad

Esta seccion sigue las categorias de RFC 6973.

Vigilancia: Las celdas tienen tamano fijo y los reles renuevan el relleno en cada salto, asi que un observador no aprende el largo

de los mensajes ni puede seguir una celda comparando bytes. En el medio optico toda celda produce la misma secuencia de 5 fragmentos (4 de 112 bytes de datos y 1 de 64), asi que el largo de las tramas no revela el largo del mensaje.

Datos almacenados: Ningun componente DEBE guardar bitacoras con direcciones IP, direcciones fisicas o identificadores de usuario. Los nodos no escriben nada a disco salvo la semilla (si se usa --llave); los mensajes recibidos viven en memoria con TTL. El Faro solo guarda contadores y una tabla de 8 duplicados en RAM. El Worker guarda mensajes publicos (200 por sala, 7 dias), blobs del buzon (TTL de hasta 72 horas, borrados al leerse) y el directorio (10 minutos por entrada).

Correlacion: La direccion fisica del Faro rota con un intervalo aleatorio. El ID de nodo no aparece en el medio optico salvo dentro de las celdas, y las celdas solo llevan IDs de un tramo. Los IDs de circuito se eligen al azar en cada tramo.

Identificacion: El remitente no se identifica ante ningun rele ni ante el destinatario. El ALIAS es opcional y elegido por el remitente. Los nombres del directorio son seudonimos.

Uso secundario: El edge no tiene datos de usuarios que reutilizar: el limite de peticiones usa un hash de la IP con la fecha y no lo persiste.

Divulgacion: Los mensajes privados solo los puede abrir el destinatario (SealedBox). El contenido del buzon muerto solo lo abre quien tenga el secreto.

Exclusion: Cualquier persona puede retirar su nodo del directorio dejando de publicarlo: la entrada vence en 10 minutos.

Boton de panico: Al presionarlo, el Faro DEBE sobrescribir el secreto guardado (en el Arduino, la EEPROM con 0xFF y luego con 0x00; en la Raspberry, el archivo con ceros antes de borrarlo), generar una direccion fisica nueva, borrar los buffers, las estadisticas y la tabla de duplicados, enviar el evento PANICO e indicarlo visualmente (los LEDs parpadean y el LED rojo queda encendido). El nodo, al recibirlo (o con el comando "panico" de la consola), DEBE borrar sus llaves, circuitos, rutas, directorio y buzon, y detenerse.

Riesgo que se acepta: el proveedor del edge sigue viendo los metadatos de red (IP, hora, tamano y ruta de cada peticion) aunque el Worker no los guarde, y puede tener sus propios registros. Nunca ve quien origino un mensaje dentro de FaroNet: solo ve al gateway.

10. Consideraciones de seguridad

Integridad en la Capa 1: El CRC-16 detecta errores accidentales, no manipulacion. Cualquiera con un LED infrarrojo puede inyectar tramas validas con cualquier direccion de origen. La autenticidad se resuelve en FaroNet con AEAD por salto, firmas Ed25519 en el directorio y SealedBox de extremo a extremo.

Negacion de servicio en el medio: Un atacante puede saturar el canal con luz modulada, y el Faro reportara CANAL_OCUPADO. No hay mitigacion en la Capa 1.

Autenticacion de reles: El handshake prueba que el rele conoce la llave onion que publico en el directorio. Un rele que no la conozca no puede completar el circuito.

Secreto hacia adelante: Las llaves de cada tramo salen de un Diffie-Hellman entre llaves efimeras, que se descartan al cerrar el circuito.

Reproduccion y reordenamiento: Los contadores de nonce de cada capa hacen que una celda repetida, reordenada o perdida rompa el descifrado y destruya el circuito. En el edge, la marca de tiempo estrictamente creciente impide repetir peticiones firmadas.

Directorio: Una entrada alterada no verifica. Las entradas vencen en 10 minutos y se rechazan las que tienen fechas fuera de una tolerancia de 5 minutos, para limitar entradas viejas. El directorio no evita ataques Sybil.

Gateway: Solo las llaves de la lista GATEWAYS del Worker pueden publicar. Si la llave de un gateway se filtra, hay que quitarla de la lista y volver a desplegar.

Secreto del Faro: LEER_SECRETO entrega la semilla a quien tenga acceso al puerto USB del Arduino o a la Raspberry. Se asume que el nodo es de confianza. En una tarjeta SD, sobrescribir un archivo no garantiza borrar todas sus copias fisicas (la tarjeta reubica los bloques), asi que en la Raspberry el panico reduce, pero no elimina, el riesgo de recuperar el secreto. El secreto protege contra el robo del Faro despues de presionar el panico, no antes. La direccion que el Arduino genera tras un panico usa un generador no criptografico (xorshift con semilla de ruido analogico), suficiente para romper la asociacion con la direccion anterior; las rotaciones normales, y todas las direcciones en la Raspberry, usan un generador criptografico.

Abuso del edge: El limite de 60 peticiones por minuto, el tamano maximo de los mensajes, la cantidad maxima de blobs del buzon y los TTL acotan el abuso de recursos.

11. Consideraciones de IANA

Este documento no requiere acciones de IANA. Los puertos TCP de los nodos son configurables (9101 por convencion en el codigo de referencia).

12. Referencias

12.1. Referencias normativas

[RFC2119] Bradner, S., "Key words for use in RFCs to Indicate Requirement Levels", BCP 14, RFC 2119, Marzo 1997.

[RFC8174] Leiba, B., "Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words", BCP 14, RFC 8174, Mayo 2017.

[RFC8032] Josefsson, S. y I. Liusvaara, "Edwards-Curve Digital Signature Algorithm (EdDSA)", RFC 8032, Enero 2017.

[RFC7748] Langley, A., Hamburg, M. y S. Turner, "Elliptic Curves for Security", RFC 7748, Enero 2016.

[RFC7693] Saarinen, M-J. y J-P. Aumasson, "The BLAKE2 Cryptographic Hash and Message Authentication Code (MAC)", RFC 7693, Noviembre 2015.

[RFC4287] Nottingham, M. y R. Sayre, "The Atom Syndication Format", RFC 4287, Diciembre 2005.

12.2. Referencias informativas

[RFC6973] Cooper, A., et al., "Privacy Considerations for Internet Protocols", RFC 6973, Julio 2013.

[RFC7322] Flanagan, H. y S. Ginoza, "RFC Style Guide", RFC 7322, Septiembre 2014.

[RFC9110] Fielding, R., Nottingham, M. y J. Reschke, "HTTP Semantics", STD 97, RFC 9110, Junio 2022.

[RFC1058] Hedrick, C., "Routing Information Protocol", RFC 1058, Junio 1988.

[TOR] Dingledine, R., Mathewson, N. y P. Syverson, "Tor: The Second-Generation Onion Router", USENIX Security Symposium, 2004.

[NTOR] Goldberg, I., Stebila, D. y B. Ustaoglu, "Anonymity and one-way authentication in key exchange protocols", Designs, Codes and Cryptography, 2013.

[XCHACHA] Arciszewski, S., "XChaCha: eXtended-nonce ChaCha and AEAD_XChaCha20_Poly1305", draft-irtf-cfrg-xchacha, 2020.

[COBS] Cheshire, S. y M. Baker, "Consistent Overhead Byte Stuffing", IEEE/ACM Transactions on Networking, 1999.

[LIBSODIUM] "libsodium documentation", https://doc.libsodium.org.

[WORKERS] Cloudflare, "Cloudflare Workers documentation", https://developers.cloudflare.com/workers/.

Apendice A. Constantes

Constante	Valor
Velocidad del Faro (defecto / rango)	2000 / 500-20000 b/s
Trama: maximo / carga maxima	128 / 114 bytes
Reintentos por defecto (rango)	3 (0 a 7)
Ranura / SIFS / DIFS	24 / 12 / 32 bits
Ventana de contencion	8 a 64 ranuras
Serial nodo-Faro	115200 8N1, COBS
Celda / cabecera / cuerpo	512 / 26 / 486 bytes
TTL inicial de una celda	16
ID de nodo	8 bytes
Fragmentos por celda en el Faro	5 (112 bytes c/u)
Costo TCP / Faro	1 / 4
Metrica infinita	16
Saltos por circuito	3 a 5 (3 por defecto)
Sobre (privado y publico)	391 bytes
Texto en claro del mensaje privado	343 bytes
Vigencia de una entrada del directorio	10 min (max. 1 h)
TTL de mensajes privados	24 h
Blob del buzon / TTL	1024 B / 1 min-72 h
Ventana de la firma del gateway	60 s
Subida del directorio al edge	cada 5 min
Limite de peticiones	60 por minuto

Apendice B. Medicion de tasa y BER

Procedimiento: dos Faros enfrentados, con luz ambiental normal, a

tres distancias. El Faro emisor envia N tramas de carga conocida; el receptor, con la opcion "entregar tramas con CRC malo", compara bit a bit.

```
tasa efectiva = bits de carga recibidos correctamente / tiempo
BER           = bits erroneos / bits recibidos
```

Estado: estas mediciones NO se realizaron. Cada Faro se probo por separado, pero no se llego a comunicar dos Faros entre si antes de la entrega, asi que no hay resultados de tasa efectiva ni de BER. La herramienta ("faro medir emisor" y "faro medir receptor") esta implementada en la biblioteca DispositivoLuzAdaptador.

Direcciones de los autores

David Zamora
Instituto Tecnologico de Costa Rica
Escuela de Computacion

Diego Zamora
Instituto Tecnologico de Costa Rica
Escuela de Computacion